

Course Name: Principles of Programming Languages

Class: S.E Computer Engineering

Course In-charge: Prof. Payal Rajput

Course Outcome

On completion of the course, learners should be able to

CO1: Make use of basic principles of programming languages.
CO2: Develop a program with Data representation and Computations.
CO3: Develop programs using Object Oriented Programming language : Java.
CO4: Develop application using inheritance, encapsulation, and polymorphism.
CO5: Demonstrate Multithreading for robust application development.
CO6: Develop a simple program using basic concepts of Functional and Logical programming paradigm.

Principles of Programming Languages

- Principles of programming languages refer to the fundamental concepts and design choices that influence how programming languages are structured, how they function, and how programmers interact with them.

- These principles form the backbone of programming languages, and understanding them is key to becoming an effective programmer.

Principles of Programming Languages

Why these principles matter ??

- These principles help you **understand** how a programming language works and guide you in writing **clean, efficient, and bug-free code**.
- They ensure that your programs are **logical**, easy to **maintain**, and can **scale** as needed.



Unit-I
Fundamentals of Programming

On completion of the unit, learners should be able to

CO1: Make use of basic principles of programming languages.

UNIT 1 FUNDAMENTALS OF PROGRAMMING

The **fundamentals of programming languages** are the building blocks that allow you to communicate with computers and create software.

❖ Why Study Programming Languages?

- To improve your ability to develop effective algorithm.
- To improve your use of your existing programming languages.
- To increase your vocabulary of useful programming constructs.
- To allow a better choice of programming language.
- To make it easier to learn a new language.
- To make it easier to design a new language.

HISTORY OF PROGRAMMING LANGUAGES

In 1960s most programming could be divided into four basic programming models:

1. **Scientific Applications**
   - Scientific applications often require large numbers of floating-point computations due to the nature of the problems they address.
   - FORTRAN: Early language for scientific/mathematical computations.
   - MATLAB: Used for numerical analysis and visualization.
2. **Business Applications**
   - Large data processing applications designed to run on big iron mainframes.
   - COBOL: Legacy language for business data processing
   - SQL: Essential for database management and data manipulation.

HISTORY OF PROGRAMMING LANGUAGES

3. Artificial Intelligence

- Artificial Intelligence is an area in which computations are symbolic rather than numeric.
- LISP: Symbolic processing and recursion for AI research.
- Prolog: Logic programming for AI applications.

4. Systems Programming

- It is system software consisting of collection of operating system, tools and infrastructure that allow applications to run on computers.
- C, C++ are generally preferred for system programming. Others like **Rust**, **Go** is also used for OS and embedded systems.



High level language

↓

Assembley language

↓

Machine Language

↓

Computer Hardware

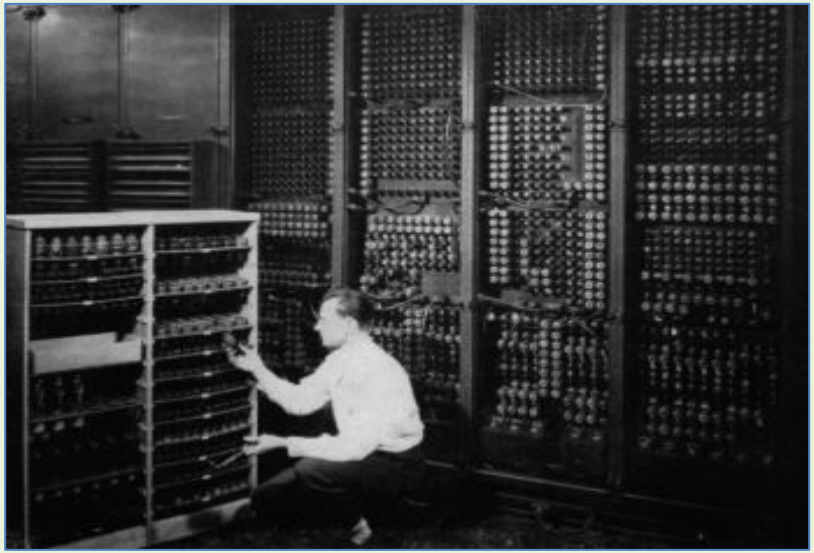
Early Beginnings (1940s - 1950s):

- **Machine Code**: Binary (0s and 1s) used by early computers.
- **Assembly Language**: First step toward human-readable code.

High-Level Languages (1950s - 1960s):

- **Fortran (1957)**: First major high-level language for scientific computing.
- **COBOL (1959)**: Designed for business applications.
- **Lisp (1958):** Developed for artificial intelligence, Lisp introduced recursion and symbolic computation.

ENIAC



UNIVAC



1970s - Rise of C and Structured Programming:

- **C (1972)**: Became the foundation for many modern operating systems.
- **Pascal (1970)**: Designed for teaching structured programming.

1980s - Object-Oriented Programming:

- **C++ (1985)**: Introduced OOP, influencing modern languages.
- **Perl (1987)**: Became popular for web scripting.



Modern Languages & Emerging Trends (1990s - Today)

- **1990s - The Internet & Scripting**:
  - **Java (1995)**: Platform-independent, widely used for web apps.
  - **JavaScript (1995)**: Revolutionized client-side web programming.
  - **Python (1991)**: Focused on readability, became widely used in web, data science, and AI.
- **2000s & Beyond - Advanced & Domain-Specific Languages**:
  - **C# (2000)**: Developed by Microsoft for enterprise applications.
  - **Go (2009)**: Focused on simplicity and scalability for web apps.
  - **Swift (2014)**: Apple's modern language for iOS/macOS development.

- **Emerging Trends**:
  - **AI and Automation**: More languages supporting AI-driven development (e.g., Python).
  - **Concurrency & Parallelism**: Languages like Go and Rust focus on performance for modern applications.







ROLE OF PROGRAMMING LANGUAGES

Programming languages play a fundamental role in software development and computing systems.

1. Facilitating Communication Between Humans and Computers
2. Solving Real-World Problems
3. Enabling Software Development
4. Enabling Cross-Platform Development
5. Optimizing Performance
6. Supporting Concurrent & Parallel Programming



ATTRIBUTES OF A GOOD LANGUAGE

1. Clarity, simplicity & unity
2. Orthogonality
3. Naturalness for the application
4. Support for abstraction
5. Ease of program verification
6. Programming environment
7. Portability of programs
8. Cost of use



SYNTAX & SEMANTICS

- **Syntax** refers to the rules that define the structure or form of valid programs written in a particular programming language.
- **Semantics** refers to the meaning or behavior of a program — essentially what the code does when it runs.
- syntax ensures that the code is written according to the language's rules, while semantics defines the logic or behavior that will result from executing the code.
- Example for syntax:

```
print('Hello, world!")
```

Output: Syntax error. As mixed double and single quote.

SYNTAX & SEMANTICS

- EXAMPLE: In **C**, using an uninitialized variable like:

```
int x;
printf("%d", x);
```

This leads to undefined behavior (a semantic error), as the variable x hasn't been assigned a value.

Progressive Education Society's
Modern College of Engineering, Pune-05.
DEPARTMENT OF COMPUTER ENGINEERING

PROGRAMMING ENVIRONMENT

A **programming environment** refers to the tools and software systems used by developers to write, test, and debug their code.

The choice of environment depends on factors such as the programming language being used, the complexity of the project, and the developer's preferences.

1) **IDE** (VSCode, JetBrains, IntelliJ IDEA, PyCharm)
2) **Compiler/Interpreter** (GCC, JVM, NodeJS)
3) **Version Control** (Git, SVN)
4) **Frameworks & libraries** (React, Angular, Vue, Django, spring)
5) **Command line tools** (Terminal, Shell scripting, package manager)

EFFECTS ON LANGUAGE DESIGN:

Programming environment have affected language design primarily in 2 major areas:

1) Separate Compilation
2) Testing/ Debugging

Separate Compilation: A method allowing independent compilation of program modules, enhancing modularity and efficiency.

- Separate compilation becomes difficult when we compile the one sub-program and it needs the information of other sub-program or shared data objects.

So, to solve this problem certain languages has adopted certain features as below:

1. Extern : can be used to indicate that particular data is used from other sub-program.
2. Scoping: is used to hide the names. One sub-program can be contained in another sub-program and name of outer sub-program are known to separately compiled sub-program
3. Inheritance: is used to share the data in object oriented programming.

Testing and Debugging: Many programming languages incorporate features that facilitate testing and debugging, enhancing the development process. These features are:

1. Set breakpoint : Pause execution at specific lines of code to inspect variable states.
   - Execute code line-by-line to observe behavior and identify issues.
   - Examine the values of variables during execution to understand program flow.
2. Execution Trace : is a powerful debugging technique that involves recording the sequence of operations executed by a program during its runtime.
   - This includes capturing function calls, variable values, and control flow, providing developers with insights into how their code behaves in real-time.
3. Assertions: It is a conditional expression which is inserted as separate statement in program.



Environment Frameworks:

- A framework is a pre-written codebase that provides a structured foundation for software development, simplifying the process and enhancing productivity.
- Borland & JBuilder are developing environment for Java.
- JBuilder: JBuilder was one of the first integrated development environments (IDEs) for Java.
- Modern programming languages makes use of VSCode, .NET for developing the applications using Visual Basic, C#, Jscript, .NET

Language	Framework	Description
Java	Spring	Framework for enterprise applications with dependency injection.
	Hibernate	ORM framework for simplifying database interactions.
Python	Django	High-level web framework for rapid development.
	Flask	Lightweight framework for small to medium applications.
PHP	Laravel	Elegant framework for web application development.
	CodeIgniter	Lightweight framework for easy web app setup.
JavaScript	React	Library for building reusable UI components.
	Angular	Platform for building mobile and desktop web applications.
Ruby	Ruby on Rails	MVC framework emphasizing convention over configuration.
.NET	ASP.NET Core	Cross-platform framework for modern web applications.

INTRODUCTION TO PROGRAMMING PARADIGM

A **programming paradigm** is a fundamental style or approach to programming, which dictates the way a programmer organizes and processes data to solve problems.

There are several key programming paradigms, and some of the most important ones include:

1. **Imperative Programming**
2. **Declarative Programming**
3. **Object-Oriented Programming (OOP)**
4. **Functional Programming**
5. **Logic Programming**
6. **Concurrent Programming**

4 IMPORTANT PROGRAMMING PARADIGM

Paradigm	Key Focus	Key Concepts	Example Languages
Procedural	Step-by-step instructions	Functions, control flow, state changes	C, Python, JavaScript, Go
Object-Oriented	Organizing code into objects	Classes, objects, inheritance, polymorphism, encapsulation	Java, Python, C++, Ruby
Functional	Functions and immutability	Pure functions, higher-order functions, immutability	Haskell, Lisp, Scala, Elixir
Logic/Rule-Based	Defining facts and rules	Facts, rules, inference, backtracking	Prolog, Mercury, CLIPS

IMPACT OF PROGRAMMING PARADIGM

The **impact of programming paradigms** on software development is significant, shaping how problems are solved, how code is written, and the overall quality and maintainability of software systems.

1. Problem- solving
2. Software Design

IMPACT OF PROGRAMMING PARADIGM ON PROBLEM SOLVING

Procedural Programming: Solves problems step-by-step, ideal for straightforward tasks.

Object-Oriented Programming (OOP): Breaks problems into objects, making complex real-world problems easier to manage.

Functional Programming: Solves problems using functions that focus on transforming data without changing its state.

Declarative Programming: Focuses on describing *what* needs to be done, allowing the system to figure out the *how*.

IMPACT OF PROGRAMMING PARADIGM ON SOFTWARE DESIGN

Procedural Programming: Leads to simple designs, but may struggle with scalability and maintenance as complexity grows.

Object-Oriented Programming (OOP): Encourages reusable objects, making software more modular, maintainable, and scalable.

Functional Programming: Promotes immutability and statelessness, making code easier to reason about and test.

Declarative Programming: Makes code concise and readable, but may be less flexible in some scenarios.

CHARACTERISTICS OF GOOD PROGRAM



Fig: Characteristics of good program

IMPACT OF MACHINE ARCHITECTURES

- In developing programming language, the architecture of software influences the design of a language in two ways:

1) The underlying computer on which programs written in the language will execute
2) The execution model, or virtual computer, that supports that language on the actual hardware

Operation of a Computer:

A computer consists of 6 major components that corresponds to programming language:

1. **Data**: A computer must provide various kinds of elementary data items & data structures to be manipulated.

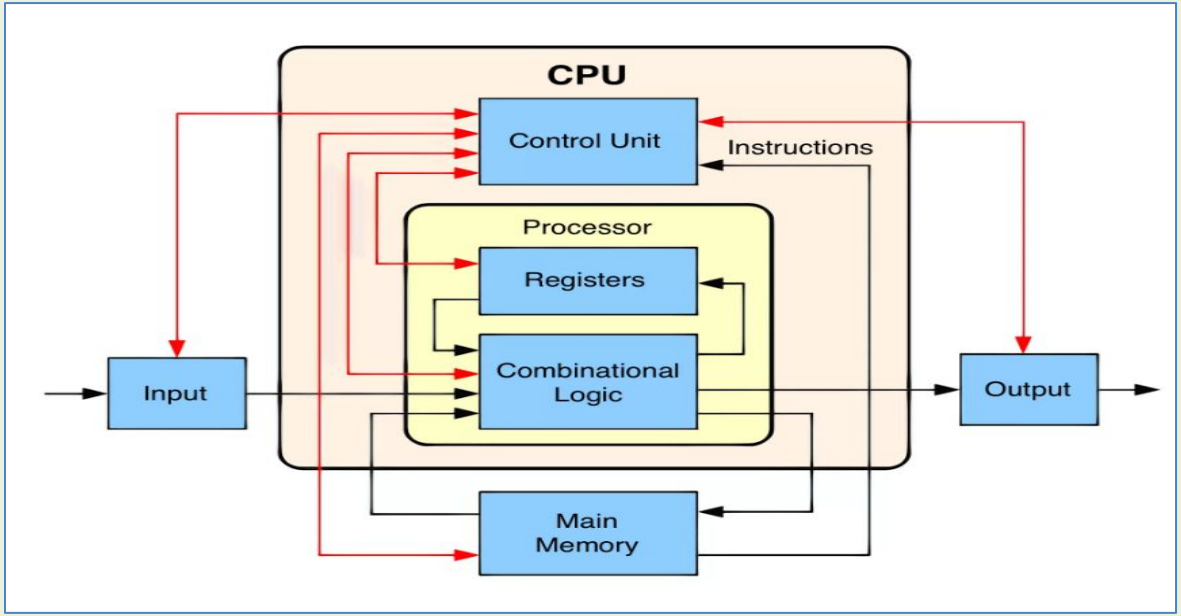


Fig: Data Storage Components



Three major data storage components are:

- Main memory, High-speed registers & External Files.

2. **Primitive operations**: A computer contains set of built-in operations like addition, subtraction,etc

- They are also used for controlling the sequence of execution of programming language.

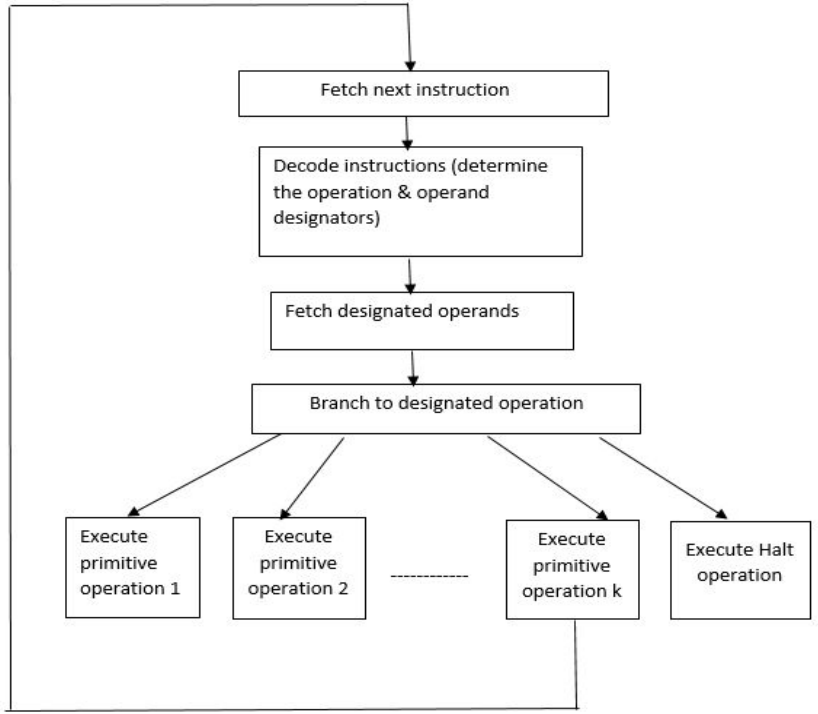
3. **Sequence Control**: The mechanism of controlling the flow of execution of programming language is called as sequence control.

- Primitive operations are used for sequence control. These operations transfer the control flow from one memory location to another.

4. **Data access**: The controlling mechanism ensures that the right data and instructions are available at the right time to execute a program.

- An operand is the data that the operation (like addition or subtraction) will be performed on.
- Each machine instruction must specify the operands. Operands might be in main memory or in a register.
- To execute a given instruction, the system must know what operands are involved and how to access them.

Fig: Program interpretation & Execution



5. **Storage Management**: It refers to the systematic control and handling of various types of memory and storage resources.

- There are 3 types of storage components: (i) Main Memory (ii) High Speed Registers (iii) External Files.
- Main Memory: Stores data and programs that are currently being used or executed.

  - Ensures smooth multitasking by swapping data in and out of RAM.

- High-Speed Registers: Located directly within the CPU for storing intermediate data during instruction execution.

  -Used for storing operand values, results, and instruction pointers during processing.

- External Files: Used to store data, programs, and files long-term.

  - Essential for large data storage when RAM is insufficient.

6. **Operating Environment:** Computer provides the mechanism by which external program helps in execution of application or user program which constitutes the programming environment.

- Operating environment consists of I/O devices & peripheral storage.

Virtual Computers & Binding Times

Virtual Computers:

- A **virtual computer** is an abstraction layer that allows users or applications to interact with a **simplified or idealized machine** without worrying about the underlying hardware complexities.
- It is the concept of creating virtualized versions of physical computer systems. Allowing you to run multiple operating systems or instances of an OS on a single physical machine.



Binding Times:

- Binding refers to the process of associating program variables and instructions with specific data, addresses, or resources during the execution of a program.
- The time at which choice of binding occurs is called binding times.
- There are various classes of binding types:

1. **Execution Time:** Bindings that occur during the execution of a program are referred to as dynamic binding or runtime binding.

- At entry to sub-program: Binding takes place between actual to formal parameters. Even binding between variables & values can be there.
- At arbitrary point during program execution: During this time variables are assigned with values.

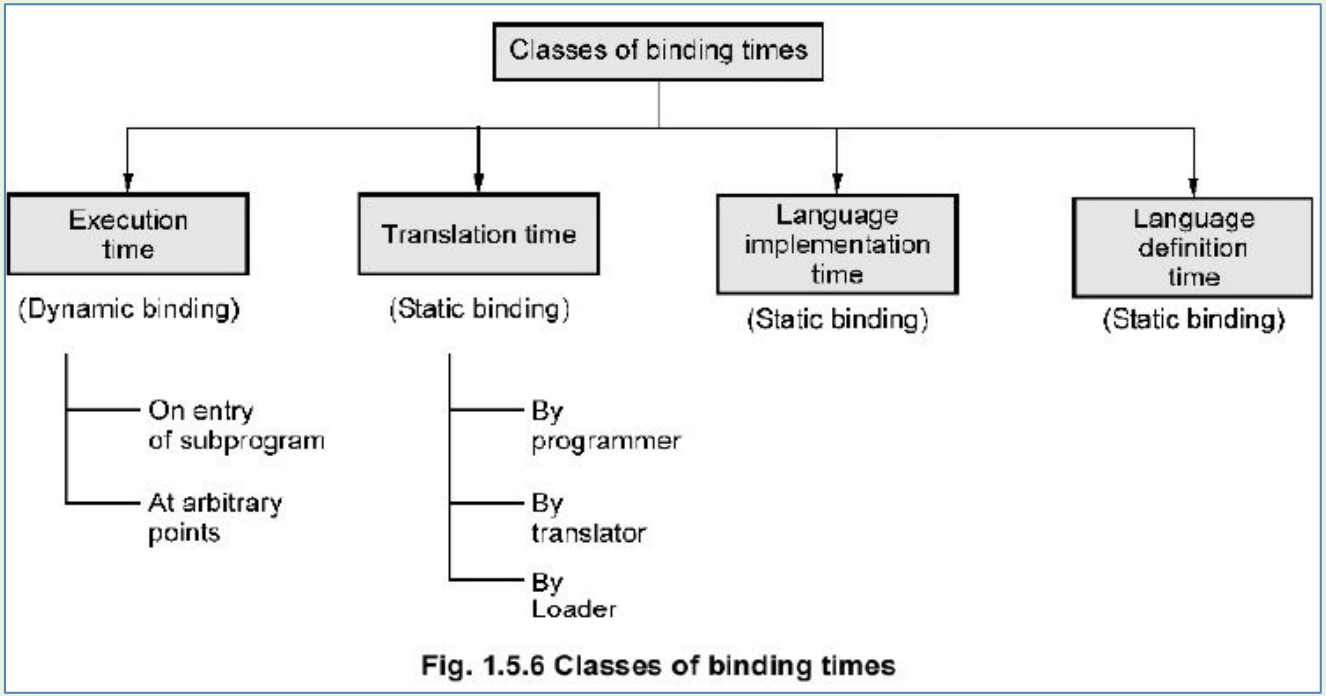


Fig. 1.5.6 Classes of binding times



Translation Time: Translation time refers to the phase in which source code is converted into executable code. Translation time can involve choices made by both programmers and loaders:

- Programmer Choices: Involve static binding decisions made at compile time, impacting how code is structured and optimized.
- Loader Choices: Involve dynamic binding decisions made at runtime, allowing for flexibility in method resolution and resource linking.

Language Implementation Time: It refers to the phase during which specific attributes of programming constructs are bound to their representations in a programming language.

- This includes defining how data types, operations, and other elements are represented and managed by the compiler or interpreter.

Language definition time: Most programming language structures are established at the time the language is defined, resulting in bindings that adhere to the predefined program structure.

- For example, in Python, if a function is defined as:

  In python

```
def add(a, b):
    return a + b
```

- The binding of the function add to its parameters a and b occurs based on the structure defined at that time. Whenever add is called, it will always expect two arguments, following the structure established in its definition.



Early Binding & Late Binding:

- Early binding and late binding are two fundamental concepts in programming that determine how method calls are resolved.
- They primarily differ in the timing of when the method to be executed is determined—either at compile time or runtime.

Early Binding	Late Binding
It happens at compile time.	It happens at run time.
The method definition and the method call are linked during the compile time. This happens when all information needed to call a method is available at the compile time.	The method definition and a method call are not bound until run time.
It is more efficient	It is more flexible.

IMPORTANT QUESTIONS

1. Why is it important to study programming languages, and how do they influence software development?
2. How do programming paradigms like procedural, object-oriented, functional, and logic-based differ in solving problems and structuring code?
3. What is the impact of machine architectures on programming languages, and how do factors like CPU, memory, and binding times affect program execution?
4. What is the concept of virtual computers, and how do binding times (static vs dynamic) affect the execution and performance of a program?

THANK YOU!!!